

Spring Boot Application Execution Flow

1. Introduction

When a Spring Boot application starts, many internal activities happen before it becomes ready to accept requests.

A simplified execution flow is:

```
main()
  ↓
SpringApplication.run()
  ↓
ApplicationContext Created
  ↓
Auto Configuration
  ↓
Component Scanning
  ↓
Bean Definitions Registered
  ↓
Beans Created
  ↓
Dependency Injection
  ↓
Bean Initialization
  ↓
ApplicationContext Ready
  ↓
Embedded Server Starts
  ↓
Application Ready
  ↓
HTTP Requests Processed
```

To understand this properly, students should know:

- ApplicationContext
- IoC
- Dependency Injection
- Spring Beans
- Bean Lifecycle
- Bean Scopes
- Auto Configuration

- Embedded Server
 - Runtime Request Processing
-

2. Starting a Spring Boot Application

A typical Spring Boot application starts from the `main()` method.

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            EmployeeApplication.class,
            args
        );
    }
}
```

The important statement is:

```
SpringApplication.run(EmployeeApplication.class, args);
```

This starts the Spring Boot application.

3. Step 1 – `main()` Method Executes

Execution begins from:

```
public static void main(String[] args)
```

Flow:

```
JVM
↓
main()
```

↓
`SpringApplication.run()`

At this point, the Spring container has not yet completed initialization.

4. Step 2 – `SpringApplication.run()`

`SpringApplication.run()` bootstraps the Spring Boot application.

Conceptually it performs tasks such as:

- Preparing the application environment
- Creating the appropriate `ApplicationContext`
- Loading application configuration
- Applying initializers
- Registering listeners
- Loading bean definitions
- Refreshing the context
- Starting the web server when applicable

Simplified:

```
SpringApplication.run()
    ↓
Prepare Environment
    ↓
Create ApplicationContext
    ↓
Load Configuration
    ↓
Refresh Context
    ↓
Start Application
```

5. What Is `ApplicationContext`?

`ApplicationContext` is the main Spring container used in modern Spring applications.

It is responsible for:

- Creating beans

- Managing beans
- Injecting dependencies
- Managing bean lifecycle
- Reading configuration
- Publishing events
- Accessing resources
- Accessing environment properties
- Supporting internationalization
- Integrating with Spring features such as AOP and transactions

Simple definition:

ApplicationContext is the Spring IoC container that creates, configures, manages, and provides application objects called Spring Beans.

6. ApplicationContext vs BeanFactory

`BeanFactory` is the basic IoC container.

`ApplicationContext` provides BeanFactory functionality plus additional enterprise features.

```
BeanFactory
  ↓
Basic Bean Management
  ↓
IoC + DI

ApplicationContext
  ↓
BeanFactory Features
  +
Events
Resources
Environment
Internationalization
AOP Integration
Transaction Integration
```

In modern Spring Boot applications, we normally work with `ApplicationContext`.

7. Which ApplicationContext Does Spring Boot Create?

The exact type depends on the application.

For a servlet-based Spring Boot web application, Spring Boot commonly creates a servlet-aware application context.

Conceptually:

```
Spring Boot Application
      ↓
Application Type Detected
      ↓
ApplicationContext Created
```

For a normal Spring MVC web application:

```
Servlet Web ApplicationContext
```

For a reactive application:

```
Reactive Web ApplicationContext
```

For a non-web application:

```
Regular ApplicationContext
```

8. Step 3 – Environment Is Prepared

Spring Boot creates an Environment containing configuration properties.

These may come from:

- application.properties
- application.yml
- Environment variables
- JVM system properties

- Command-line arguments
- Profile-specific property files
- External configuration

Example:

```
server.port=8081

spring.datasource.url=jdbc:postgresql://localhost:5432/employeedb

spring.datasource.username=postgres

spring.datasource.password=admin
```

9. Configuration Sources

Spring Boot supports externalized configuration.

Example sources:

```
application.properties
+
application.yml
+
Environment Variables
+
Command-Line Arguments
+
System Properties
```

This allows application configuration to change without changing Java source code.

10. @SpringBootApplication

The main application class usually contains:

```
@SpringBootApplication
```

This is a convenience annotation that combines:

```
@SpringBootApplication
|
|— @SpringBootConfiguration
|— @EnableAutoConfiguration
|— @ComponentScan
```

These three concepts are central to Spring Boot startup.

11. @SpringBootConfiguration

This identifies the main configuration class for the Spring Boot application.

It is based on Spring's:

```
@Configuration
```

This allows bean definitions and configuration to be registered.

12. @EnableAutoConfiguration

This tells Spring Boot to configure the application automatically based on:

- Classpath dependencies
- Existing beans
- Configuration properties
- Application type
- Other conditions

Example:

If Spring Boot sees:

```
spring-boot-starter-web
```

it can configure:

- Spring MVC
- DispatcherServlet
- Embedded Tomcat

- Jackson
 - HTTP message converters
-

13. What Is Auto Configuration?

Auto Configuration means:

Spring Boot automatically creates commonly required configuration based on the dependencies and configuration available in the application.

Conceptually:

```
Check Classpath
  ↓
Check Existing Beans
  ↓
Check Properties
  ↓
Evaluate Conditions
  ↓
Create Required Beans
```

14. Example of Auto Configuration

Suppose you add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-jpa
  </artifactId>
</dependency>
```

and configure:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/employeedb
spring.datasource.username=postgres
spring.datasource.password=admin
```


Spring Boot can configure infrastructure around:

```
DataSource
  ↓
JPA
  ↓
Hibernate
  ↓
EntityManagerFactory
  ↓
TransactionManager
```

15. @ComponentScan

Component scanning finds application classes that should become Spring Beans.

Common annotations:

```
@Component
@Service
@Repository
@Controller
@RestController
@Configuration
```

Example package:

```
com.example.employee
|
├─ EmployeeApplication.java
├─ controller
├─ service
├─ repository
└─ config
```

If the main class is in:

```
com.example.employee
```

Spring Boot scans that package and its subpackages by default.

16. Step 4 – Bean Definitions Are Registered

Spring first discovers bean definitions.

Example classes:

```
@RestController
public class EmployeeController {
}
```

```
@Service
public class EmployeeService {
}
```

```
@Repository
public class EmployeeRepository {
}
```

Conceptually:

```
Component Scanning
    ↓
Find Components
    ↓
Create Bean Definitions
    ↓
Register With Container
```

A bean definition is metadata that tells Spring how a bean should be created.

17. What Is a Spring Bean?

A Spring Bean is an object managed by the Spring container.

Example:

```
@Service
public class EmployeeService {
}
```

Spring creates something conceptually similar to:

```
EmployeeService object
    ↓
Managed by ApplicationContext
    ↓
Spring Bean
```

18. IoC – Inversion of Control

IoC means that control of object creation and management is transferred from application code to the Spring container.

Traditional Java:

```
EmployeeRepository repository =
    new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repository);
```

Here the application controls object creation.

With Spring:

```
@Service
public class EmployeeService {
}
```

Spring creates the object.

Conceptually:

Traditional Java

Application

↓

Creates Objects

↓

Manages Objects

With Spring:

Spring

Application

↓

Spring Container

↓

Creates Objects

↓

Manages Objects

That is Inversion of Control.

19. Why IoC Is Useful

IoC provides:

- Loose coupling
 - Easier testing
 - Better maintainability
 - Easier object management
 - Clear dependency management
 - Reusable components
-

20. Dependency Injection

Dependency Injection is one of the main techniques used to implement IoC.

Suppose:

```
EmployeeController
  ↓
EmployeeService
  ↓
EmployeeRepository
```

Instead of manually creating these dependencies, Spring injects them.

Example:

```
@RestController
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(
        EmployeeService employeeService) {

        this.employeeService = employeeService;
    }
}
```

Spring finds the `EmployeeService` bean and injects it.

21. Constructor Injection

Constructor injection is generally recommended.

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}
```

Advantages:

- Dependencies are explicit.
 - Dependencies can be `final`.
 - Easy to test.
 - Helps enforce required dependencies.
 - No `@Autowired` required when there is only one constructor.
-

22. Setter Injection

Example:

```
@Service
public class EmployeeService {

    private EmployeeRepository employeeRepository;

    @Autowired
    public void setEmployeeRepository(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}
```

Setter injection may be useful for optional or replaceable dependencies.

23. Field Injection

Example:

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;
}
```

Field injection works, but constructor injection is generally preferred because dependencies are more explicit and unit testing is easier.

24. Step 5 – Beans Are Instantiated

After bean definitions are registered, Spring creates bean objects.

Example:

```
Bean Definitions
    ↓
EmployeeController
EmployeeService
EmployeeRepository
    ↓
Objects Created
```

For singleton beans, Spring normally creates them during ApplicationContext initialization unless configured for lazy initialization.

25. Singleton Bean

The default Spring bean scope is:

```
singleton
```

This means:

One bean instance per Spring container for a given bean definition.

Example:

```
@Service
public class EmployeeService {
}
```

By default:

```
ApplicationContext
    ↓
EmployeeService
```

↓
One Managed Bean Instance

26. Bean Scopes

Spring supports several scopes.

Scope	Description
Singleton	One instance per bean per Spring container
Prototype	New instance each time requested
Request	One instance per HTTP request
Session	One instance per HTTP session
Application	One instance per ServletContext
WebSocket	One instance per WebSocket session

27. Singleton Scope

Default scope:

```
@Service
public class EmployeeService {
}
```

Equivalent conceptually to:

```
@Scope("singleton")
```

Use singleton scope for:

- Stateless services
- Controllers
- Repository components
- Utility services

Most Spring application beans are singleton scoped.

28. Prototype Scope

Example:

```
@Component
@Scope("prototype")
public class ReportGenerator {
}
```

Every time the bean is requested, Spring creates a new instance.

```
getBean()
  ↓
Instance 1

getBean()
  ↓
Instance 2
```

Useful when objects contain temporary or stateful data.

29. Request Scope

One object is created for each HTTP request.

Example:

```
@Component
@RequestScope
public class RequestContext {
}
```

Request 1:

```
RequestContext Object 1
```

Request 2:

RequestContext Object 2

30. Session Scope

One bean is created for each HTTP session.

```
@Component
@SessionScope
public class UserSession {
}
```

Useful for session-related information.

31. Application Scope

One bean exists for the lifetime of the ServletContext.

Example:

```
@Component
@ApplicationScope
public class ApplicationStatistics {
}
```

Useful for shared web-application-level data.

32. Step 6 – Dependency Injection Happens

Once Spring creates the required beans, it resolves their dependencies.

Suppose:

```
EmployeeController needs EmployeeService

EmployeeService needs EmployeeRepository
```

Spring resolves:

```
EmployeeRepository Bean
    ↓
Injected Into
EmployeeService Bean
    ↓
Injected Into
EmployeeController Bean
```

Result:

```
Controller
    ↓
Service
    ↓
Repository
```

All objects are managed by the container.

33. Dependency Resolution Example

```
@Repository
public class EmployeeRepository {
}
```

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(
        EmployeeRepository repository) {

        this.repository = repository;
    }
}
```

```
@RestController
public class EmployeeController {
```

```

private final EmployeeService service;

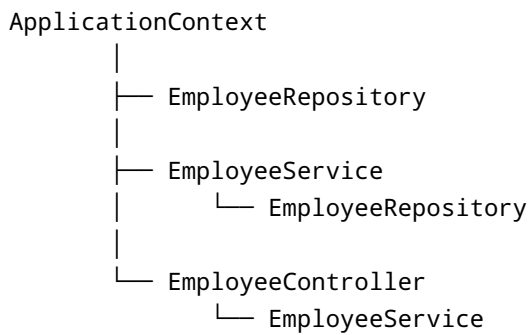
public EmployeeController(
    EmployeeService service) {

    this.service = service;
}
}

```

Spring understands this dependency graph.

34. Dependency Graph

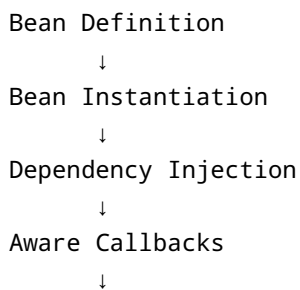


Spring creates the dependencies in the necessary order.

35. Bean Lifecycle

Every Spring Bean follows a lifecycle.

Simplified lifecycle:



```
BeanPostProcessor Before Initialization
↓
Initialization
↓
BeanPostProcessor After Initialization
↓
Bean Ready
↓
Bean Used
↓
Destruction
```

36. Bean Lifecycle – Detailed Flow

A simplified Spring lifecycle is:

1. Instantiate Bean
2. Populate Properties / Inject Dependencies
3. Execute Aware Interfaces
4. BeanPostProcessor
 postProcessBeforeInitialization()
5. @PostConstruct
6. InitializingBean.afterPropertiesSet()
7. Custom init method
8. BeanPostProcessor
 postProcessAfterInitialization()
9. Bean Ready for Use
10. @PreDestroy
11. DisposableBean.destroy()
12. Custom destroy method

37. @PostConstruct

`@PostConstruct` runs after dependencies have been injected.

Example:

```
@Service
public class EmployeeService {

    @PostConstruct
    public void init() {

        System.out.println(
            "EmployeeService initialized"
        );
    }
}
```

Use cases:

- Initial setup
- Cache initialization
- Validation
- Loading startup information

38. @PreDestroy

`@PreDestroy` runs before the bean is destroyed.

Example:

```
@PreDestroy
public void cleanup() {

    System.out.println(
        "Cleaning EmployeeService resources"
    );
}
```

Possible uses:

- Closing resources
- Releasing connections
- Cleaning caches
- Shutdown logic

39. Aware Interfaces

Spring provides Aware interfaces that let beans access container infrastructure.

Examples:

```
BeanNameAware  
ApplicationContextAware  
EnvironmentAware  
ResourceLoaderAware
```

Example:

```
@Component  
public class MyBean  
    implements ApplicationContextAware {  
  
    @Override  
    public void setApplicationContext(  
        ApplicationContext context) {  
  
        System.out.println(context);  
    }  
}
```

These are useful in special infrastructure cases, but normal application code should usually prefer dependency injection.

40. BeanPostProcessor

`BeanPostProcessor` allows custom logic around bean initialization.

Two important methods:

```
postProcessBeforeInitialization()
```

and:

```
postProcessAfterInitialization()
```

Conceptually:

```
Bean Created
  ↓
Dependencies Injected
  ↓
Before Initialization Processor
  ↓
Initialization
  ↓
After Initialization Processor
  ↓
Bean Ready
```

Spring itself uses BeanPostProcessors internally for many advanced features.

41. Proxies and BeanPostProcessor

Spring may wrap beans with proxy objects for features such as:

- AOP
- Transactions
- Security
- Async processing
- Caching

Example:

```
@Transactional
public void saveEmployee() {
}
```

The actual injected bean may be a Spring-generated proxy that surrounds the original object.

Conceptually:


```
Caller
  ↓
Spring Proxy
  ↓
Transaction Logic
  ↓
EmployeeService
```

42. ApplicationContext Refresh

A critical part of startup is:

```
ApplicationContext.refresh()
```

Conceptually, refresh performs work such as:

- Preparing the BeanFactory
- Registering BeanPostProcessors
- Instantiating singleton beans
- Injecting dependencies
- Initializing beans
- Publishing context events

When refresh completes successfully, most application singleton beans are ready.

43. ApplicationContext Ready

After the context is refreshed:

```
ApplicationContext
  ↓
Beans Created
  ↓
Dependencies Injected
  ↓
Beans Initialized
  ↓
Application Components Ready
```

At this stage, business components can be used.

44. Embedded Server Startup

For a typical Spring Boot MVC web application, Spring Boot starts an embedded server such as:

Tomcat

Other supported options can include:

Jetty
Undertow

The server starts during application context initialization for a web application.

45. Embedded Server Flow

```
ApplicationContext
    ↓
Web Server Factory
    ↓
Embedded Tomcat
    ↓
Port Opened
    ↓
Application Accepts HTTP Requests
```

Default port:

8080

Can be changed:

`server.port=8081`

46. When Is the Application Ready?

After startup completes successfully, Spring Boot logs a message similar to:

```
Started EmployeeApplication
```

At this stage:

- `ApplicationContext` is initialized.
- Singleton beans are ready.
- Dependencies are injected.
- Embedded server is running.
- HTTP endpoints can receive requests.

47. Complete Startup Flow

Students can remember this flow:

1. JVM starts
2. `main()` executes
3. `SpringApplication.run()` executes
4. Environment prepared
5. `ApplicationContext` created
6. Configuration loaded
7. Auto configuration applied
8. Component scanning performed
9. Bean definitions registered
10. Beans instantiated
11. Dependencies injected
12. Bean lifecycle callbacks executed

13. ApplicationContext refreshed
14. Embedded server started
15. Application becomes ready

48. Runtime Request Flow

After startup, the application begins processing requests.

Example:

```
GET /employees/101
```

Runtime flow:

```
Client
  ↓
Embedded Tomcat
  ↓
Spring Security Filters
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

Response:

```
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
```

```
↓  
DispatcherServlet  
↓  
Tomcat  
↓  
Client
```

49. DispatcherServlet

`DispatcherServlet` is Spring MVC's Front Controller.

It handles requests after the web server passes them into the Spring MVC application.

Its responsibilities include:

- Receiving requests
- Finding controller methods
- Coordinating request processing
- Calling handlers
- Handling responses

50. Controller

Example:

```
@RestController  
@RequestMapping("/employees")  
public class EmployeeController {  
  
    private final EmployeeService service;  
  
    public EmployeeController(  
        EmployeeService service) {  
  
        this.service = service;  
    }  
  
    @GetMapping("/{id}")  
    public EmployeeResponseDto getEmployee(  
        @PathVariable Long id) {
```

```
        return service.getEmployee(id);
    }
}
```

Controller responsibility:

```
Handle HTTP Request
    ↓
Call Service
    ↓
Return HTTP Response
```

51. Service

Example:

```
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(
        EmployeeRepository repository) {

        this.repository = repository;
    }

    public EmployeeResponseDto getEmployee(
        Long id) {

        // Business logic
        return null;
    }
}
```

Responsibilities:

- Business logic
- Validations
- Transactions
- Coordination
- Data transformation

52. Repository

Example:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
}
```

Repository responsibility:

```
Service
  ↓
Repository
  ↓
JPA / Hibernate
  ↓
Database
```

53. Request Scope During Runtime

Request-scoped beans are created when an HTTP request begins.

```
HTTP Request 1
  ↓
Request Bean 1
  ↓
Request Completed
  ↓
Bean Destroyed
```

Another request creates another instance.

54. Singleton Beans During Runtime

Most Controller, Service, and Repository beans are singleton scoped.

Example:

```
Application Startup
  ↓
EmployeeService Bean Created
  ↓
Same Bean Reused
  ↓
Request 1
Request 2
Request 3
```

Because singleton beans are shared, they should normally be stateless or thread-safe.

55. Important Singleton Warning

Avoid keeping request-specific mutable information in singleton beans.

Bad example:

```
@Service
public class EmployeeService {

    private String currentEmployeeName;
}
```

Many requests may use the same service instance concurrently.

Instead, keep request data in:

- Method parameters
 - Local variables
 - DTOs
 - Request-scoped objects when actually needed
-

56. IoC vs DI

Students often confuse IoC and DI.

IoC

IoC is the broader principle.

Spring Container controls object creation.

DI

DI is one mechanism Spring uses to implement IoC.

Spring injects required objects into other objects.

Simple:

IoC = Who controls objects?

Answer:
Spring Container

DI = How are dependencies provided?

Answer:
Spring injects them

57. ApplicationContext vs IoC

ApplicationContext is the actual Spring container.

IoC is the design principle it implements.

IoC
↓
Concept

ApplicationContext
↓
Implementation / Container

58. ApplicationContext vs Bean

```
ApplicationContext
    ↓
Container

Bean
    ↓
Object Managed by Container
```

Example:

```
ApplicationContext
|
├─ EmployeeController Bean
├─ EmployeeService Bean
├─ EmployeeRepository Bean
└─ DataSource Bean
```

59. Common Spring Boot Annotations

Annotation	Purpose
<code>@SpringBootApplication</code>	Main Boot application configuration
<code>@Component</code>	Generic Spring Bean
<code>@Service</code>	Business layer bean
<code>@Repository</code>	Data access layer bean
<code>@RestController</code>	REST controller
<code>@Controller</code>	MVC controller
<code>@Configuration</code>	Configuration class
<code>@Bean</code>	Explicit bean creation
<code>@Autowired</code>	Dependency injection
<code>@Qualifier</code>	Select a bean
<code>@Primary</code>	Preferred bean

Annotation	Purpose
<code>@Scope</code>	Bean scope
<code>@PostConstruct</code>	Initialization callback
<code>@PreDestroy</code>	Destruction callback
<code>@Transactional</code>	Transaction management

60. Configuration Example

```
server.port=8080

spring.application.name=employee-service

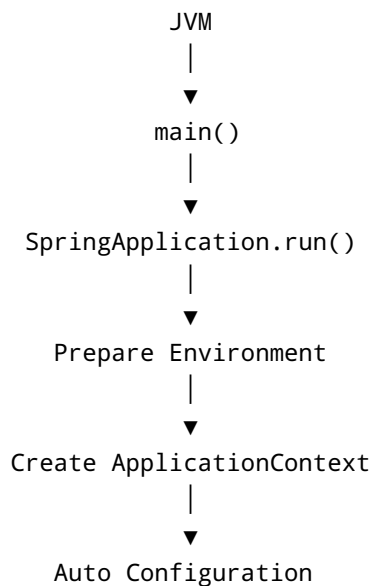
spring.datasource.url=jdbc:postgresql://localhost:5432/employeedb

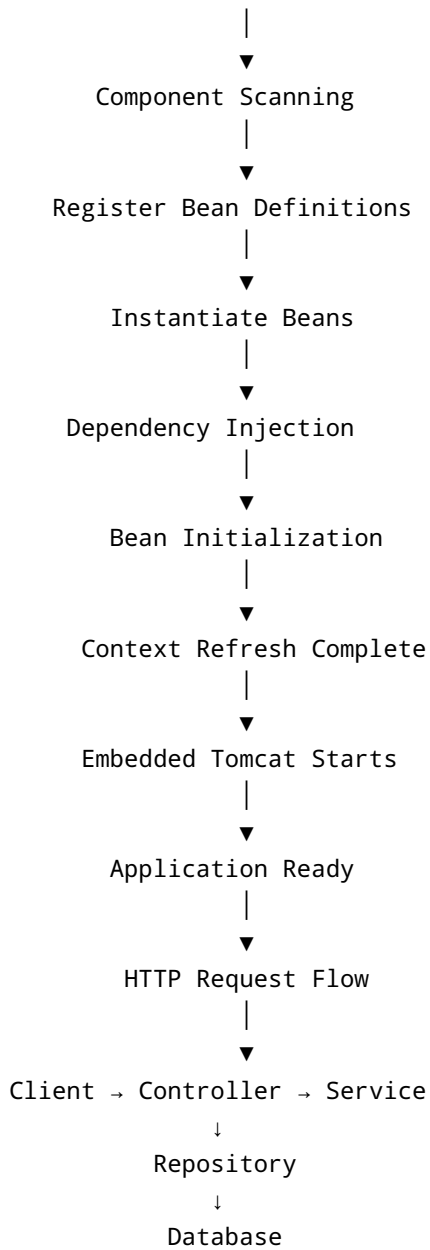
spring.datasource.username=postgres

spring.datasource.password=admin
```

Spring Boot loads these values into the Environment.

61. Full Execution Architecture





62. Startup Flow vs Runtime Flow

This difference is very important.

Startup Flow

Happens once when the application starts.

```
main()
↓
ApplicationContext
↓
Beans
↓
DI
↓
Bean Initialization
↓
Server
```

Runtime Flow

Happens for every request.

```
Client
↓
Tomcat
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
↓
Database
```

63. Interview Question: What Happens When Spring Boot Starts?

A good answer:

When a Spring Boot application starts, the main method calls `SpringApplication.run()`. Spring Boot prepares the Environment, creates the `ApplicationContext`, applies auto configuration, performs component scanning, registers bean definitions, instantiates beans, injects dependencies, executes bean lifecycle callbacks, refreshes the `ApplicationContext`, and starts the embedded web server. Once initialization completes, the application is ready to process HTTP requests.

64. Interview Question: What Is ApplicationContext?

ApplicationContext is Spring's IoC container. It creates, configures, manages, and provides Spring Beans, performs dependency injection, manages bean lifecycle, reads environment and configuration data, and supports features such as events, resources, AOP, and transaction management.

65. Interview Question: Difference Between IoC and DI

IoC is the principle where control of object creation and management is transferred to the Spring container. Dependency Injection is a mechanism used to implement IoC by providing a bean's required dependencies from outside rather than allowing the bean to create them itself.

66. Interview Question: Default Bean Scope

The default Spring bean scope is:

singleton

Meaning:

One bean instance per Spring container per bean definition.

67. Interview Question: Recommended Injection Type

Constructor injection is generally recommended.

Example:

```
private final EmployeeRepository repository;

public EmployeeService(
    EmployeeRepository repository) {

    this.repository = repository;
}
```

Advantages:

- Required dependencies are clear.
- Supports immutability.
- Easy to test.
- No field reflection required.
- Works naturally with `final` fields.

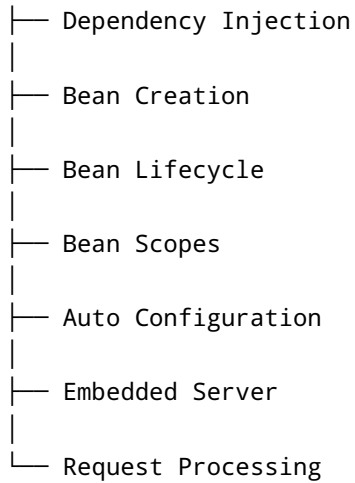
68. Key Benefits

Spring Boot's execution model provides:

- Loose coupling through IoC and DI
- Managed object lifecycle
- Centralized configuration
- Auto configuration
- Easier testing
- Embedded server
- Modular architecture
- Production-ready integration
- Faster application setup
- Easier maintenance

69. Important Concepts to Remember

```
Spring Boot
|
├─ SpringApplication.run()
|
├─ ApplicationContext
|
├─ IoC
|
```



70. Simple Student Memory Flow

Remember this line:

```
START
  ↓
CONTAINER
  ↓
BEANS
  ↓
DEPENDENCIES
  ↓
INITIALIZATION
  ↓
SERVER
  ↓
REQUESTS
```

Expanded:

```
main()
  ↓
SpringApplication.run()
  ↓
ApplicationContext
  ↓
Bean Creation
  ↓
```



```
Dependency Injection
  ↓
Bean Initialization
  ↓
Tomcat Starts
  ↓
Application Ready
```

71. Final Summary

The complete Spring Boot execution can be divided into three major areas.

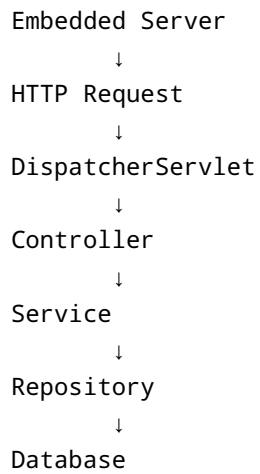
Phase 1 – Bootstrapping

```
main()
  ↓
SpringApplication.run()
  ↓
Environment
  ↓
ApplicationContext
```

Phase 2 – Spring Container Initialization

```
Auto Configuration
  ↓
Component Scan
  ↓
Bean Definitions
  ↓
Bean Creation
  ↓
Dependency Injection
  ↓
Bean Lifecycle
  ↓
Context Ready
```

Phase 3 – Application Runtime



The most important idea is:

Spring Boot starts by creating and preparing the `ApplicationContext`. The `ApplicationContext` acts as the IoC container, creates Spring Beans, injects their dependencies, manages their lifecycle and scope, and once initialization is complete, Spring Boot starts the embedded server so the application can process requests.